Data Science

Foundations of Machine Learning

Sylvain BENOIT & Arthur THOMAS

Dauphine | PSL
UNIVERSITÉ PARIS

What is Machine Learning?

What does "learning" mean?

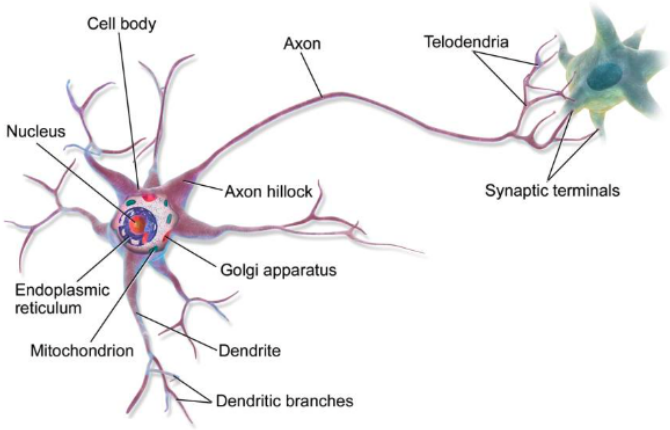
- Tom Mitchell's (1997) definition:

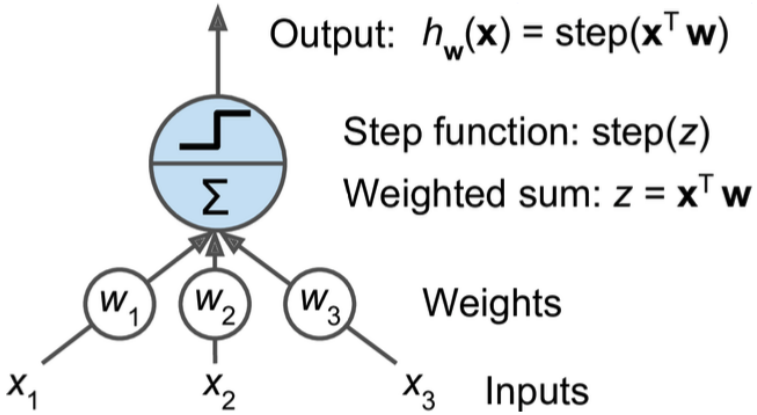
  A computer program is said to **learn** *from experience $\mathcal{E}$ concerning some task $\mathcal{T}$ and performance measure $\mathcal{P}$, if its performance at task $\mathcal{T}$, as measured by $\mathcal{P}$, improves with experience $\mathcal{E}$.*

- **Example:** ML program for spam filtering
  - Task $\mathcal{T}$: identify spams in new emails
  - Experience $\mathcal{E}$: training data (email dataset)
  - Performance $\mathcal{P}$: percentage of accurately classified emails
- Remarks:
  - The performance measure $\mathcal{P}$ is not necessarily unique
  - The above performance measure $\mathcal{P}$ is called **accuracy**
  - Other performance measures could be considered: percentage of spam the program can identify among actual spam messages (**recall**), percentage of correct predictions when the program identifies an email as a spam message (**precision**)

Deep Learning (DL) - I

- DL is a type of ML that mimics how the human brain is functioning through the modelling of neuron behaviour and interactions
- Biological neurons receive signals from other neurons through synapses in the form of short electric impulses. When a neuron receives several signals that exceed a certain threshold, it emits its electrical impulse

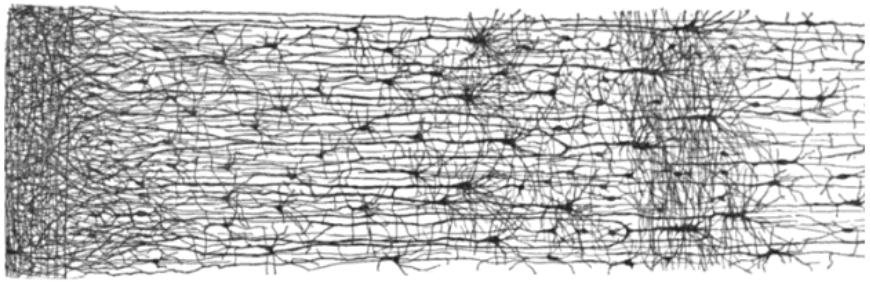




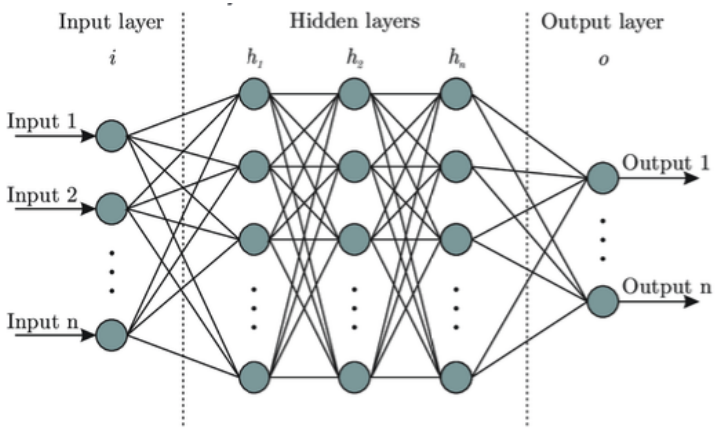
Source: A. Géron, "Hands-on Machine Learning with Scikit Learn, Keras & TensorFlow", 2nd ed., O'REILLY (2019)

Deep Learning (DL) - II

- In a DL model, artificial neurons are connected through a network, hence the name Artificial Neural Network or ANN
- As in a human brain, an ANN contains several layers of neurons
- Different types of ANN depending on how neurons are connected across the various layers of the network
- The "deep" term in DL refers to the fact that the number of neurons involved in an ANN can be huge (potentially several millions)



Source: A. Géron, "Hands-on Machine Learning with Scikit Learn, Keras & TensorFlow", 2nd ed., O'REILLY (2019)



Traditional programming approach

- Spam classifier (Géron, 2019)
  1. Looking at what a spam message typically looks like, you would build a list of words, names, mail address features, etc. to establish a set of rules
  2. You would write an algorithm that classifies an email as spam or ham based on the rules you established
  3. You would test your program and repeat steps 1 and 2 until the algorithm is acurate enough
- Isuues
  - Identifying spams is not an easy task $\Rightarrow$ you would end up with a long list of (potentially complex) rules
  - Cat-and-mouse problem: when spam issuers realize that their emails are systematically blocked whenever their messages contain some patterns, they will change the way their emails are written, and your spam detector will become inefficient $\Rightarrow$ you need to extend the set of rules

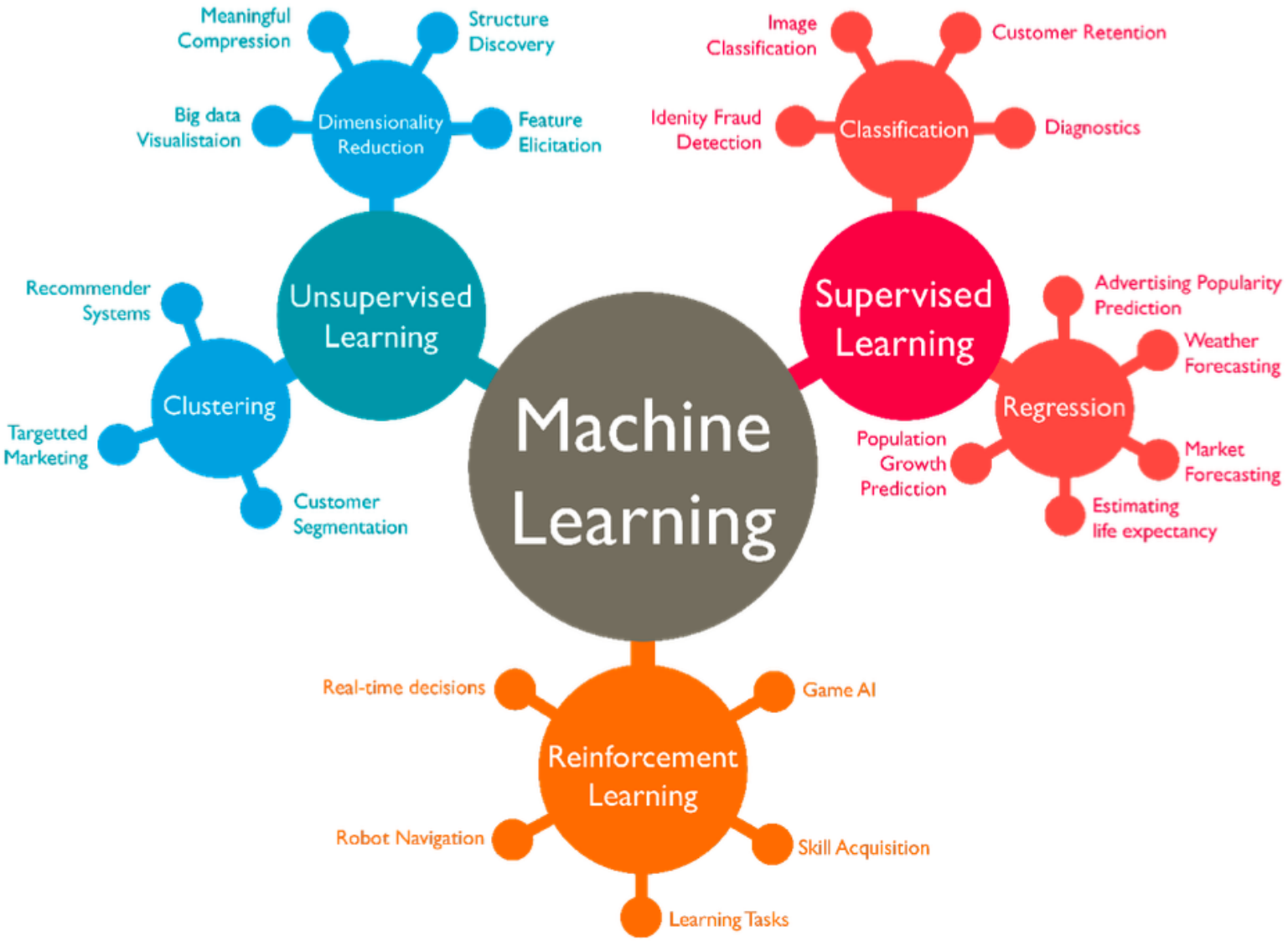
Machine Learning approach

- A ML-based spam filter will automatically learn which patterns are good predictors to identify spam messages based on the differences they detect between ham and spam examples in a (training) database
- Advantages:
  - The classification rules are inferred by the program $\Rightarrow$ no need to write explicit rules
  - The program is much shorter
  - Does not solve the cat-and-mouse problem but updating the algorithm is made easier $\Rightarrow$ catching up with the spammers simply needs to retrain the algorithm using an updated database
- Other advantages:
  - ML algorithms can be inspected (not necessarily easy for some of them) to identify what are the best predictors of a spam message $\Rightarrow$ can reveal unsuspected correlations, and new trends and help better understanding a problem
  - When applied to large amount of data, ML can uncover patterns that are not immediately apparent (data mining)

Differences with econometrics

- Econometrics is **model-centric**
  - Modeling is rooted in an underlying economic mechanism
  - Objectives: inference of data-generating process from a (random) sample of whole population, model parameter estimation ($\hat{\beta}$), and potentially predictions ($\hat{y}$) based on estimates
  - Quality of estimators is fundamental $\Rightarrow$ consistency, hypothesis tests, confidence intervals...
  - Interpretation of the model is key: Why do we get the estimated relationship and what does it mean? (XAI)
- Machine learning is **data-centric** and essentially focuses on prediction
  - $\hat{y}$ is more important than $\hat{\beta}$
  - ML is agnostic about the data-generating process
  - Interpretation of how a prediction is obtained is a second-order issue
  - Sophisticated toolkit to asses model robustness in a world where large-sample properties of the estimators are potentially out of reach

Types of Machine Learning


Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

Supervised learning

- Supervised learning is a ML type where a model is built from training on a dataset such that each data point (example / sample) is represented by both:
  - Its features, also known as descriptors, attributes or predictors
  - Its label also known as the target or the outcome
- Examples:
  - Features on appartment data: size, number of rooms, clarity, location. Label: selling price
  - Features on company accouning data: revenues, working capital, EBITDA. Label 0: the company is in business the year after. Label 1: company bankruptcy
- Objective: learn the relationship between features and labels in order to make predictions on new data
- Two broad classes of supervised learning problems
  - Label is a continuous variable: regression task
  - Label is a class (bankruptcy or not, one digit among 10 possible ones): classification task

Unsupervised learning

- In an unsupervised learning problem, the training set is unlabeled
- The objective is to build a model that allows to get a representation of the observations that conveys more information than their raw values
- Unsupervised learning can be used to:
  - Partition data into homogenous groups: clustering
  - Getting a representation of the data in a lower-dimension space compared with the original space: dimensionality reduction
- Examples:
  - Customer segmentation – Anomaly detection
  - Data compression
  - Data visualization

Semisupervised learning

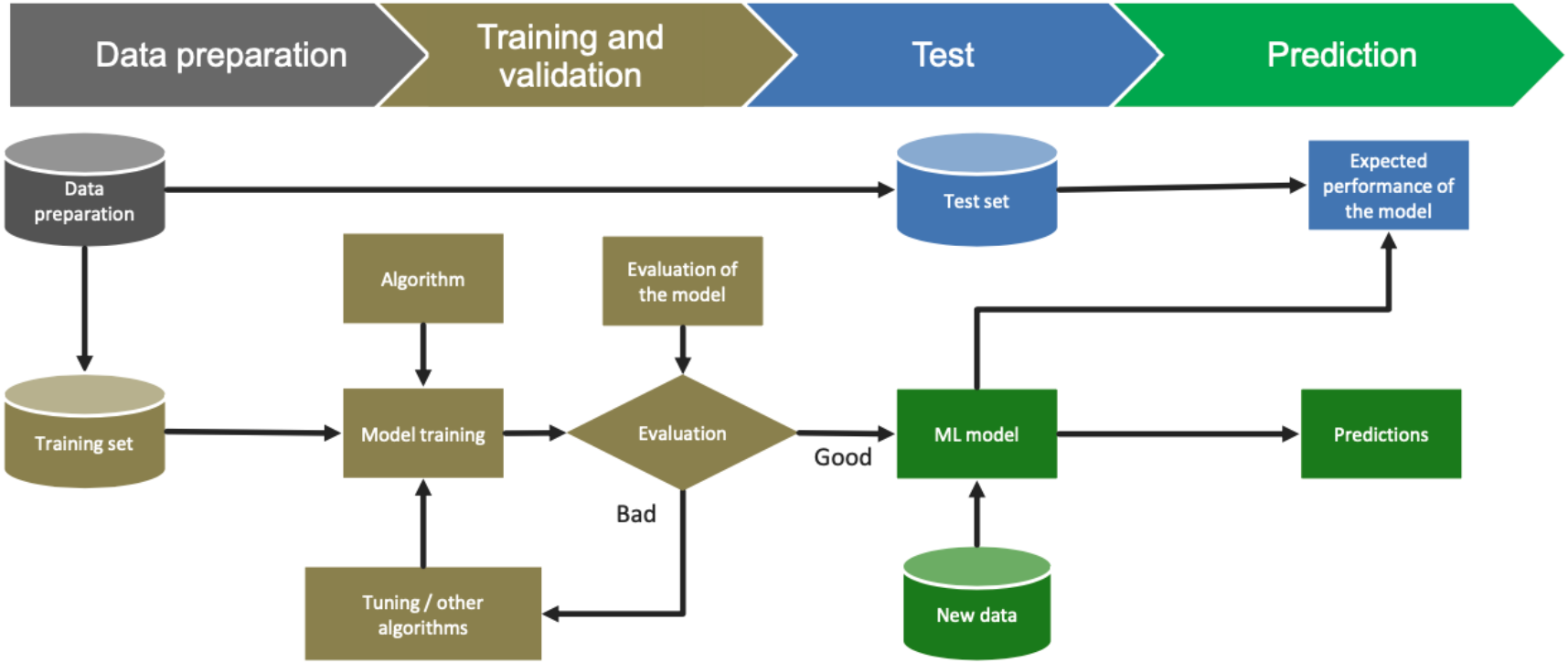
- Assigning labels to data is both time-consuming and costly
- Human performing the labelling task can make errors or can be affected by biases $\Rightarrow$ errors and biases will be reproduced by a fully supervised ML algorithm
- Semisupervided ML algorithms can help overcome these issues
- Example: Google Photos service
  - An unsupervised algorithm (typically a clustering model) recognizes that the same person shows up in different photos
  - Telling once who is that person in a photo (i.e. assigning a label) allows the system to tag her in any other photo where she shows up

Reinforcement learning

- In reinforcement learning, a system called the agent selects and performs actions. In return for these actions, it receives penalties (negative return) and rewards (positive return)
- The agent must learn by itself the best strategy, called a policy, to maximize its discounted expected utility
- Trade-off between exploitation(repeat actions that have been successful in the past) and exploration (try different actions, whose outcome is uncertain)
- Examples:
  - Vacuum-cleaning robots
  - DeepMind's AlphaGo program
  - Hide and seek game: Click here

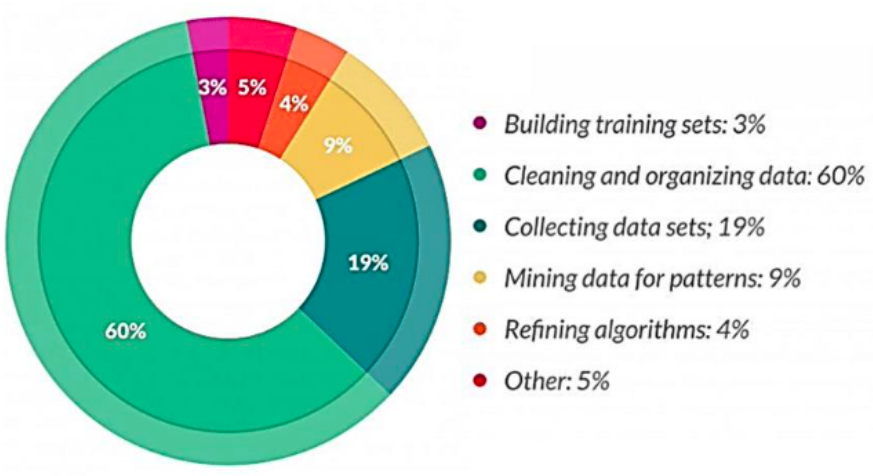
The Machine Learning process

ML process



- Algorithm: ML procedure (e.g. linear regression) run on data to produce an ML model
- ML model: output of an ML algorithm, i.e. what is learned by the ML algorithm (e.g. a vector of regression coefficients values)

Data preparation - I



Source: https://www.researchgate.net/figure/What-data-scientists-spend-the-most-time-doing-7_fig2_335577003

- Not the exciting part of an ML project but crucial: garbage in $\longrightarrow$ garbage out
- Some software facilitates the data-wrangling part of the process (e.g. Trifacta)

Data preparation - II

- Frequent issues:
  - Duplicated observations
  - Missing data
  - Heterogeneity in data reporting conventions: frequency of observations, units, date format, missing data representation, ...
  - Inconsistencies between databases
  - ...
- Some rules:
  - Get acquainted with the data to get a better understanding of what they represent and understand the way they are encoded
  - Perform exploratory analyses: number of observations, number of missing values, the average value of features, median, standard deviation, inter-quartile range, binning/clustering, correlations, ...
  - Objective: identification of useful features, outliers, ...

Data preparation - III

- Data cleaning :
  - Missing data handling: deletion or imputation
  - Duplicated data: techniques for deduplicating data
  - Handling of outliers: deletion, winsorization
  - Inconsistencies: partially automatic detection is made possible through functional dependencies (e.g. a city must always have the same zip code)
- Data transformation :
  - Feature scaling: some ML algorithms do not perform well when the input numerical attributes display very different scales, e.g. revenue vs. ratio. Normalization and standardization are two common ways to get all attributes to have the same scale
  - Aggregation: e.g. cumulate daily data to transform them into monthly data
  - Features construction: addition or replacement of some features by new features computed from existing ones (e.g. log transformation or power transformation to capture non-linearities)
  - Discretization: the transformation of numerical values into classes

Data preparation - IV

- Dimensionality reduction (if needed):
  - Objective: representation of features in a lower-dimension space to improve training
  - Techniques: feature aggregation, PCA (principal component analysis),auto-encoders
- Data anonymization: a crucial issue nowadays
  87% of the population in the United States had reported characteristics that likely made them unique based only on 5-digit ZIP, gender, date of birth (Sweeney, 2002)
  - Anonymization techniques:
    - k-anonymity: grouping of the observations within partitions such that each partition contains at least k individuals sharing the same quasi-identifier
    - l-diversity: in addition to k-anonymity, each partition must contain at least l different values of a sensitive attribute
    - t-closeness: the distribution of the sensitive attribute within any partition must be the same as its distribution over the entire dataset
    - Differential privacy: given two similar databases, an algorithm that enforces DP must behave in the same way on the two databases

Training and validation - I

- The first step of any ML project is to split the data into 3 different sets: a train set, a validation set and a test set
- The train set is used to train the model, i.e. to determine the parameters that minimize the errors
- Warning:
  - Beware of imbalanced data
    Example: bankruptcy prediction. In a typical dataset, only 6% of the firms will eventually go bankrupt. If the train set contains 94% of healthy firms, the classification algorithm will learn nothing: it will broadly predict that all companies are healthy and its accuracy will be equal to 94%!
  - If feature scaling is used: normalization or standardization must be performed without using the data from the test set or the validation set to avoid any look-ahead bias. This implies that the validation set and the test set will be normalized or standardized based on the normalization or standardization factors computed from the train set

Training and validation - II

- In the training step several competing algorithms are trained and their hyperparameters are fine-tuned
- Once the models are trained their quality is compared on the validation set using performance metrics
- Based on the performances of the competing models on the validation set, other rounds of training can be run on the train set for further fine-tuning
- Once one has come up with the final version of the candidate models, the best one is selected. Potentially, the model can be retrained using both the training set and the validation set

Test phase and beyond

- Once the best model has been selected, it has to be tested on the test set
- In general, the performance of the model will be inferior to the one it displays on the train set or the validation set
- Golden rule: never tweak the hyperparameters to make the model look better on the test set
  - Typically, adjusting the model will make it overly specialized to fit the training data
  - The "improvements" will result in larger generalization error due to overfitting
  - Eventually, the model will display poorer performance on unseen data
- Next step: deployment of the model and production phase
- Warning: a model must be monitored constantly as its performance will eventually drop $\Rightarrow$ the model will have to be trained again on fresh data

Supervised learning framework

Framing of a supervised learning problem

A supervised learning problem can be represented as follows:

- Let consider a train set $\mathcal{S} = \{(x_i, y_i)\}_{i=1,\dots,n}$ containing n examples
  - $x_i, i = 1, \dots, n$ are the features in $\mathcal{S}$ and are such that $x_i \in \mathcal{X}$
  - $y_i, i = 1, \dots, n$ are the labels in $\mathcal{S}$ and are such that $y_i \in \mathcal{Y}$
- It is assumed that there exists an unknown mapping function $\phi : \mathcal{X} \mapsto \mathcal{Y}$ such that $y_i = \phi(x_i) + \epsilon_i$ where ϵ_i is a (random) noise term
- The objective of supervised learning is to find function $h^*_\theta(x) : \mathcal{X} \mapsto \mathcal{Y}$ such that, for every pair $(x, \phi(x)) \in \mathcal{X} \times \mathcal{Y},\ h^*_\theta(x) \approx \phi(x)$

Remarks:

- From a probabilistic viewpoint, it is assumed that pairs (x_i, y_i) are realizations of a pair of random variables (X, Y) such that $Y = \phi(X) + \epsilon$, where ϵ is a noise term
- In many ML tasks, $\mathcal{X} = \mathbb{R}^p$, though other types of representations exist (binary variables, categorical variables, strings of characters, graphs, etc.)

ML tasks and labels

- A regression task refers to a Machine Learning problem where labels take on real values, i.e. $\mathcal{Y} = \mathbb{R}$
- In binary classification, labels are binary variables, and $\mathcal{Y} = \{0, 1\}$. For some algorithms (e.g. Support Vector Machines), it is mathematically more convenient to consider the label space $\mathcal{Y} = \{-1, 1\}$
- In multiclass classification, the labels take on a limited number of discrete values called classes or categories, with one value per class. The label space is $\mathcal{Y} = \{0, \ldots, C\}$

Data representation

The $\mathcal{S}$ set can be represented in matrix notation as follows:

$$X = \begin{bmatrix} x_{1,1} & x_{1,1} & \cdots & x_{1,p} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n,1} & x_{n,2} & \cdots & x_{n,p} \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Remarks:

- X is the matrix of features, also known as the design matrix
- As X is a n rows by p columns matrix, $\mathcal{S}$ contains n examples and each example has p features
- Y is the n-rows column vector of labels

Hypothesis

- **Reminder**: The objective of supervised learning is to find function $h^*_\theta(x) : \mathcal{X} \mapsto \mathcal{Y}$ such that, for every pair $(x, \phi(x)) \in \mathcal{X} \times \mathcal{Y},\ h^*_\theta(x) \approx \phi(x)$
- Question: which h?
- The hypothesis space, denoted $\mathcal{H}$, is the space of functions which describes all the models we plan to consider. This space is chosen based on priors about the task
- The choice of the hypothesis space is crucial: if $\mathcal{H}$ does not contain the appropriate function, achieving $h^*_\theta(x) \approx \phi(x)$ is impossible.

Choosing a hypothesis: illustration

- Suppose we want to predict the price y of an apartment based on its features x
- Since $y \in \mathbb{R}$ (in reality $\mathbb{R}^+$), we need a hypothesis that satisfies this constraint
- An acceptable hypothesis in this respect is $h_\theta(x_i) = \theta_0 + \sum_{j=1}^{p} \theta_j x_{i,j}$
- On the contrary, the hypothesis $h_\theta(x_i) = \frac{1}{1+exp\left(\theta_0+\sum_{j=1}^{p} \theta_j x_{i,j}\right)}$ should not be considered as its outputs are comprised of between 0 and 1. However, this hypothesis is suitable for binary classification
- Would $h_\theta(x_i) = \theta_0 + \sum_{j=1}^{p} \theta_j x_{i,j}$ be a suitable hypothesis for binary classification? Yes, if we consider that the values returned by h_θ^* that exceed a certain threshold are indicative of class 1 whereas those below that threshold are indicative of class 0

From hypothesis to cost function

- Once the h_θ hypothesis has been chosen, how do we come up with function h_θ^* such that $h_\theta^*(x) \approx \phi(x)$?
- Two additional ingredients needed:
  1. Measurement of the quality of a hypothesis: is the hypothesis that is used satisfactory (potentially optimal)? This implies specifying a cost or loss function
  2. Once a loss function has been chosen, determining h_θ^* from h_θ involves methods to minimize the training loss of a model

Cost function and risk minimization

- A cost function or loss function is a function $\ell : \mathcal{Y} \times \mapsto \mathbb{R}^+$ that measures the distance or loss between the predicted label and the true label
- Given cost function ℓ, the objective is to determine h_θ^* such that the cost computed from all possible values $x \in \mathcal{X}$ is minimal.
- Since pairs (x, y) are random draws from (X, Y), and assuming that the joint distribution of (X, Y), is $\mathbb{P}_{X,Y,h_\theta^*}$ has to minimize the expected loss, also known as the risk of model h_θ:

$$\mathcal{R}(h_\theta) = \mathbb{E}\left[\ell\left(Y, h_\theta(X)\right)\right] = \int_{\mathcal{X}\times\mathcal{Y}} \ell\left(y, h_\theta(x)\right) \mathrm{d}\mathbb{P}_{X,Y}(x, y)$$

- Differently stated, the h_θ^* function we are looking for is such that $h_\theta^* \in \underset{h\in\mathcal{H}}{\operatorname{argmin}}\ \mathbb{E}\left[\ell\left(Y, h_\theta(X)\right)\right]$

Empirical risk

- In practice, the actual risk $\mathcal{R}(h_\theta)$ cannot be computed since it would imply that all labels $\mathcal{Y}$ from $\mathcal{X}$ are known $\Rightarrow$ the actual $\mathcal{R}(h_\theta)$ will be approximated using $\mathcal{S} = \{(x_i, y_i)\}_{i=1,\dots,n}$ through the computation of empirical risk
- For a supervised learning problem, assuming n examples from $\mathcal{S} = \{(x_i, y_i)\}_{i=1,\dots,n}$ the empirical risk or training loss is defined as the estimator:

$$\mathcal{R}_n(h_\theta) \equiv L_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, h_\theta(x_i))$$

- Therefore, predictor h_θ^* obtained from empirical risk minimization is:

$$h_\theta^* \in \underset{h \in \mathcal{H}_\theta}{\operatorname{argmin}} \ \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, h_\theta(x_i))$$

Cost functions

Which cost function?

- There is a cornucopia of cost functions $\Rightarrow$ which one should be selected?
- The choice of a particular cost function is driven by the type of problem one is addressing
- Practical issues such as the speed at which the loss can be minimized or the precision of the minimization that can be achieved must be accounted for
- Most of the time the chosen cost function is convex to make the optimization problem easier to solve

Regression - I

The most common loss function for regression tasks is the quadratic loss or squared error:

Quadratic loss

$$\ell_{SE} : \mathbb{R} \times \mathbb{R} \mapsto \mathbb{R}$$
$$y, h_\theta(x) \mapsto \frac{1}{2}(y - h_\theta(x))^2$$

- Due to the squared term, the quadratic loss function heavily penalizes large errors
- One advantage of quadratic loss is that it is differentiable, which facilitates gradient computation (more on this in the next slides)
- The $\frac{1}{2}$ term shows up for mathematical convenience only and has no impact on the optimal solution

Regression - II

Large errors may be such that squared loss pays little attention to small but frequent errors

Absolute loss or Absolute error function

$$\ell_{AE} : \mathbb{R} \times \mathbb{R} \mapsto \mathbb{R}$$
$$y, h_\theta(x) \mapsto \frac{1}{2} |y - h_\theta(x)|$$

- With ℓ_{AE}, all errors are weighted on the same linear scale so that predictions close to the actual label are penalized too (though mildly)
- A potential issue is that the resulting model may perform well most of the time but make some very poor predictions

Regression - III

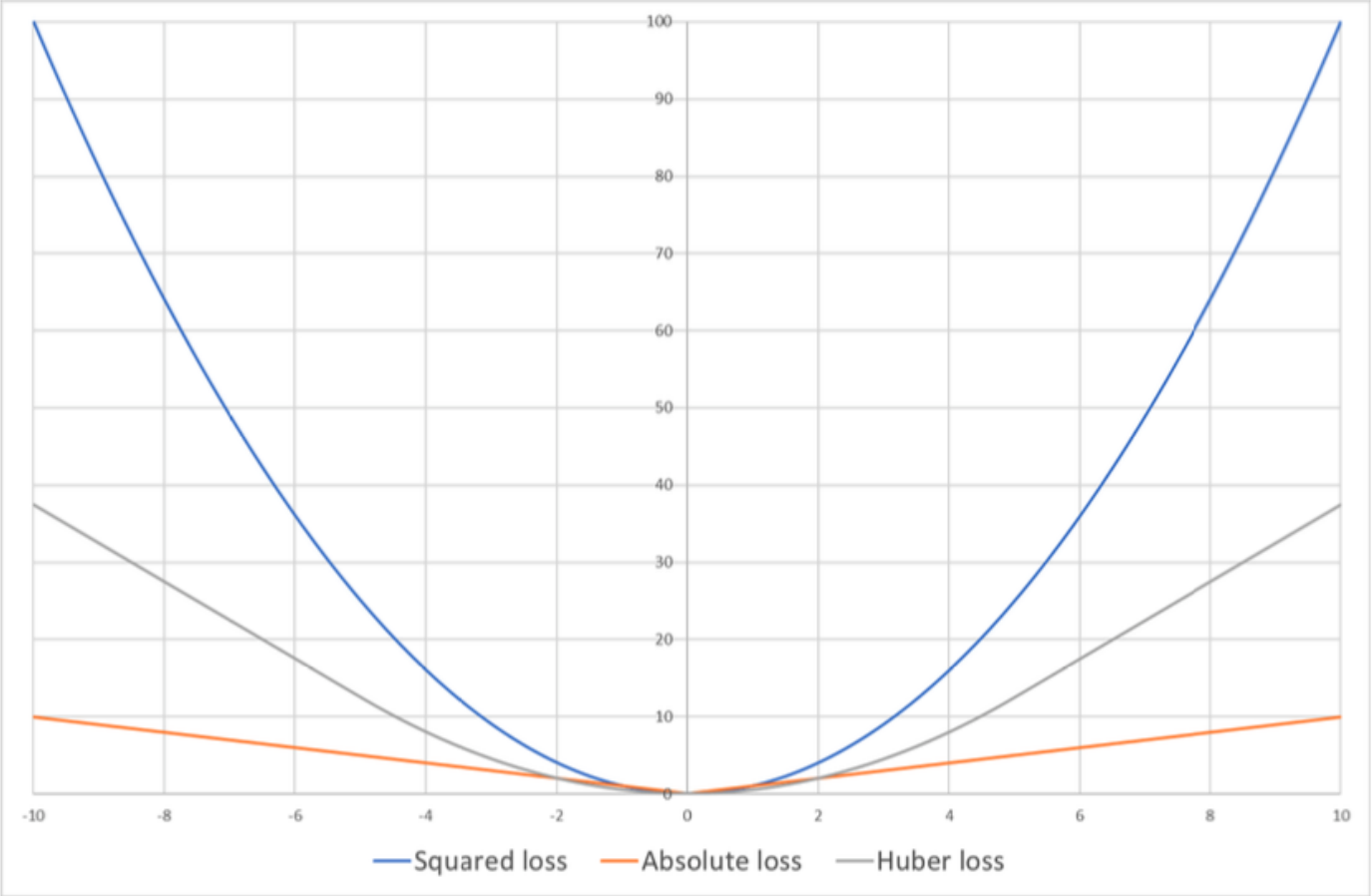
One way to combine the advantages of ℓ_{SE} and ℓ_{AE} is:

Huber loss

$$\ell_{Huber} : \mathbb{R} \times \mathbb{R} \mapsto \mathbb{R}$$
$$y, h_\theta(x) \mapsto \begin{cases} \frac{1}{2}(y - h_\theta(x))^2 \text{ if } |y - h_\theta| < \zeta \\ \zeta\,|y - h_\theta(x)| - \frac{1}{2}\zeta^2 \text{ otherwise} \end{cases}$$

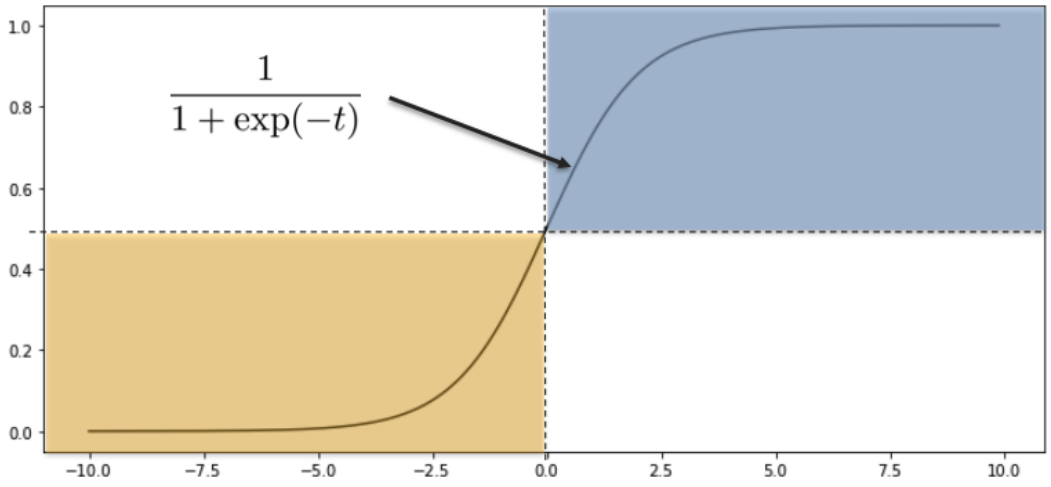
- In ℓ_{Huber}, for errors below ζ loss is computed as ℓ_{SE}, whereas it is computed as ℓ_{AE} for errors above ζ
- Intuition: using ℓ_{AE} for large errors mitigates the weight that is put on outliers while small errors are down-weighted by ℓ_{SE} to focus training on higher-error training examples
- ζ is a hyperparameter that controls the range of inliers vs. outliers. As such, it has to be tuned by the modeller (more on this in the next slides)

Graphical representation



Binary classification: logistic regression - I

- There exist several suitable cost functions to perform a binary classification task (e.g. 0/1 loss, hinge loss, ...)
- Here, we are going to focus on a binary classification algorithm known as logistic regression
- The starting point of logistic regression is the logistic function or sigmoid function $\sigma : \mathbb{R} \mapsto (0,1), t \mapsto \frac{1}{1+exp(-t)}$



Binary classification: logistic regression - II

- Since the logistic function outputs values in the interval $(0, 1)$, the output $\sigma(h_\theta(x_i))$ can be interpreted as the estimation by the model of probability $\hat{p}_i$ that the label of example i is $y_i = 1$
- Using this feature we define

Binary cross-entropy loss

$$\begin{aligned} \ell_H : &\{0,1\} \times \{0,1\} \mapsto \mathbb{R} \\ y, \sigma(h_\theta(x)) &\mapsto -y log\left[\sigma(h_\theta(x))\right] - (1-y) log\left[1 - \sigma(h_\theta(x))\right] \end{aligned}$$

- One can check that ℓ_H is an increasing function of errors:
  - if the model predicts a low $\hat{p}_i$ when $y_i = 1$, $-y_i log(\hat{p}_i)$ is large
  - if the model predicts a large $\hat{p}_i$ when $y_i = 0$, $-(1-y_i) log(1-\hat{p}_i)$ is large

Training

Definitions

- The objective of model training is to determine the θ parameter set that achieves empirical risk minimization based on hypothesis h_θ.
- For some algorithms (e.g. linear regression), closed-form formulas yield the exact solution to the empirical risk minimization problem
- For other algorithms (e.g. logistic regression) approximate solutions only are available
- A standard technique to determine an approximate solution is gradient descent

In this course we will study three implementations of the gradient descent technique:

- Batch gradient descent
- Stochastic gradient descent (SGD)
- Mini-batch gradient descent

Linear regression setup - I

- Let consider training examples $\{(x_i, y_i)\}_{i=1,\dots,n}$ where $x_i = [x_{i,1}\ x_{i,2}\ \dots\ x_{i,p}]$
- Without loss of generality, let's consider the hypothesis:

$$h_\theta(x_i) = \theta_0 + \sum_{j=1}^{p} \theta_j x_{i,j}$$

- Remarks:
  - θ_0 is called the bias term and the various $\theta_j, j = 1, \dots, p$ are called the weights
  - More complex hypotheses could be considered such as polynomial features
- Assuming squared loss is used, the expression for empirical risk is:

$$L_n(x, y, \theta) = \frac{1}{2n} \sum_{i=1}^{n} \left(y_i - \theta_0 - \sum_{j=1}^{p} \theta_j x_{I,j} \right)^2$$

Linear regression setup - II

- Let consider design matrix X, column vector of labels y, and column vector of parameters θ:

$$X = \begin{bmatrix} 1 & 1 & x_{1,1} & x_{1,1} & \dots & x_{1,p} \\ 1 & x_{2,1} & x_{2,2} & \dots & x_{2,p} & \\ \vdots & \vdots & \vdots & \ddots & \vdots & \\ 1 & x_{n,1} & x_{n,2} & \dots & x_{n,p} & \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix}$$

- With these notations, the hypothesis now expresses as $h_\theta(X) = X\theta$
- The expression for empirical risk corresponds to the Mean Squared Error (MSE) and is computed as:

$$L_n(X, y, \theta) = \frac{1}{2n} [y - X\theta]^T . [y - X\theta] \tag{1}$$

Training: closed-form solution

- Expanding (1), $L_n(X, y, \theta)$ can be re-expressed as:

$$L_n(X, y, \theta) = \frac{1}{2n}\left[y^T y - y^T X\theta - \theta^T X^T y + \theta^T X^T X\theta\right]$$

- Differentiating with respect to θ:

$$\frac{\partial L_n(X, y, \theta)}{\partial \theta} = \frac{1}{n}\left[X^T X\theta - X^T y\right]$$

- Equating with 0, and assuming that $(X^T X)$ is invertible, we get:

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

- Note that $\hat{\theta}$ in corresponds to the standard OLS estimator of θ

Issues with OLS

- The issue with the OLS estimator is that it involves the computation of the inverse of $(X^T X)$
- For very large datasets (big data), computing this inverse can be extremely costly
- Also, the $(X^T X)$ matrix may not be invertible
- An alternative to OLS estimator is gradient descent

Gradient descent: principle

- Gradient descent aims at finding the optimal $\hat{\theta}$ using an iterative procedure that updates the model parameters based on the value of the gradient of $L_n(X, y, \theta)$ with respect to θ
- The algorithm is as follows:
  1. Initialize $\theta^{(0)}$ elements with random values
  2. For all $j \in [1, p]$, repeat until convergence (or other stopping criterion):

$$\theta_j^{(k+1)} := \theta_j^{(k)} - \eta \frac{\partial L_n(X, y, \theta)}{\partial \theta_j} \left(X, y, \theta_j^{(k)}\right)$$

- η is called the learning rate. It determines the size of the downhill step
- As η is at the discretion of the modeller, it is a hyperparameter of the model
  - If it is too small, convergence will be slow and potentially will not be reached
  - If it is too high, the parameter values may bounce around their "true" value and potentially the algorithm may diverge

Gradient descent: graphical illustration

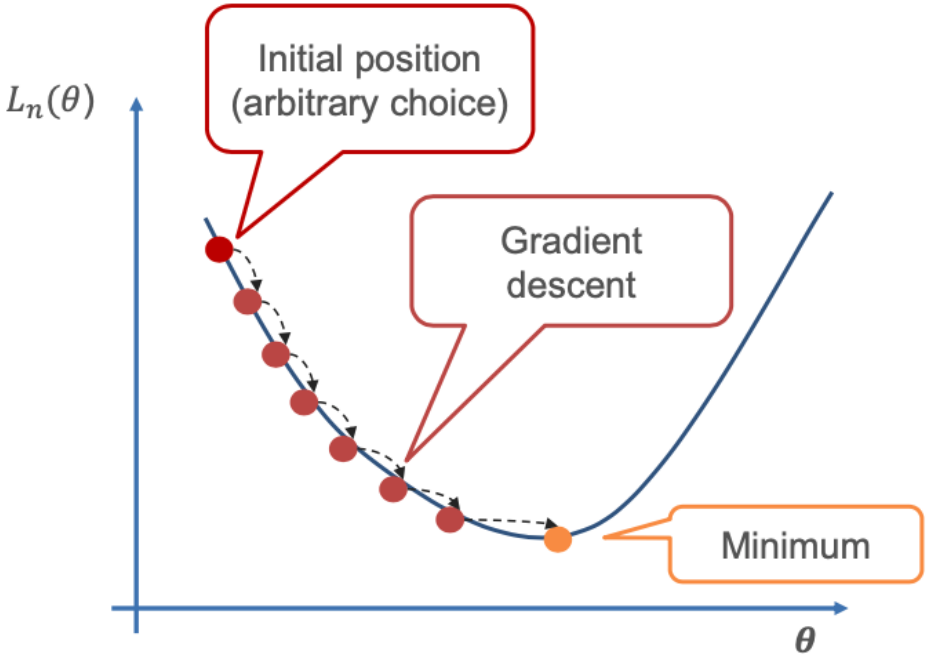


Illustration with a single parameter (θ_0)

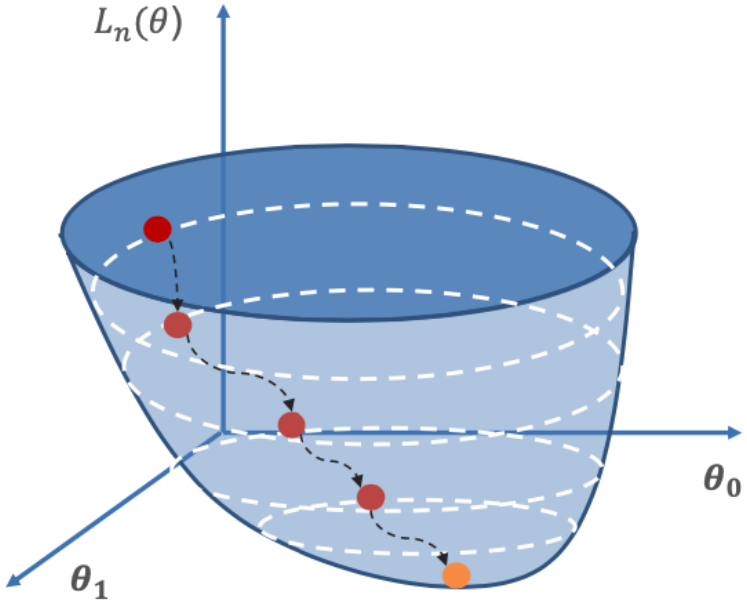


Illustration with two parameters (θ_0, θ_1)

Gradient descent for linear regression

We established the derivative of the training loss with respect to θ as:

$$\frac{\partial L_n(X, y, \theta)}{\partial \theta} = \frac{1}{n}\left[X^T X\theta - X^T y\right]$$

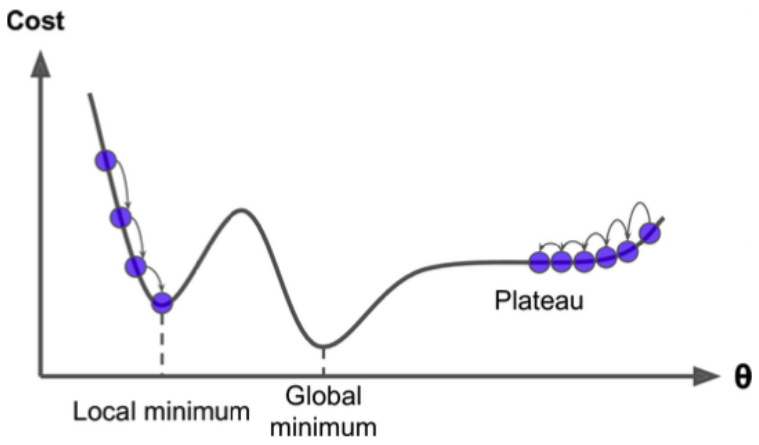
Gradient descent for linear regression is thus performed as follows:

1. Draw p random numbers to initialize $\theta^(0)$
2. Repeat until convergence or other stopping criteria:

$$\theta^{(k+1)} := \theta^{(k)} - \frac{1}{n}\eta\left[X^T X\theta^{(k)} - X^T y\right]$$

Issues with batch gradient descent

- The gradient procedure previously described is called batch gradient descent
- "batch" means that at each iteration the procedure uses all training examples to update model parameters
- If the cost function is not convex, depending on the values used to initialize gradient descent, the algorithm may converge to a local minimum
  - Another issue is related to speed: as the algorithm uses the whole training set to update parameters, it becomes extremely slow on massive datasets

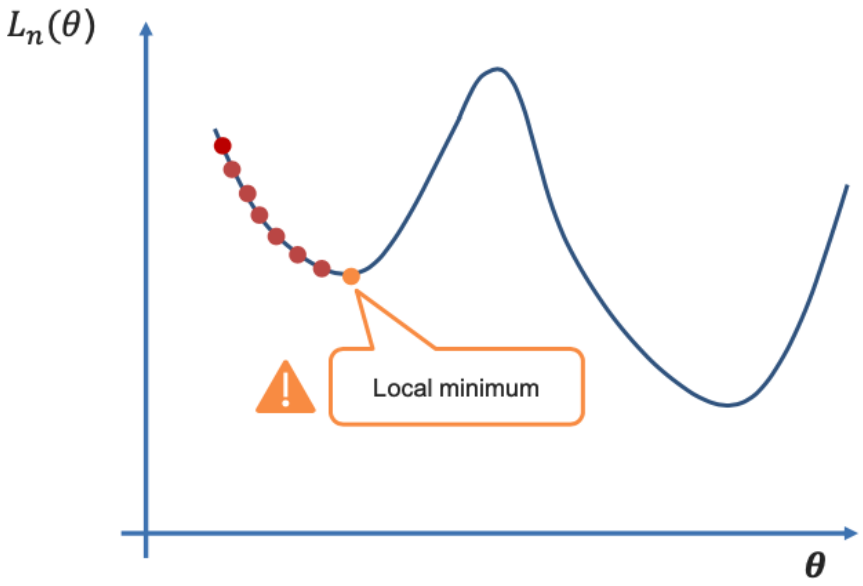


Source: A. Géron, "Hands-on Machine Learning with Scikit Learn, Keras TensorFlow", 2nd ed., O'REILLY (2019)

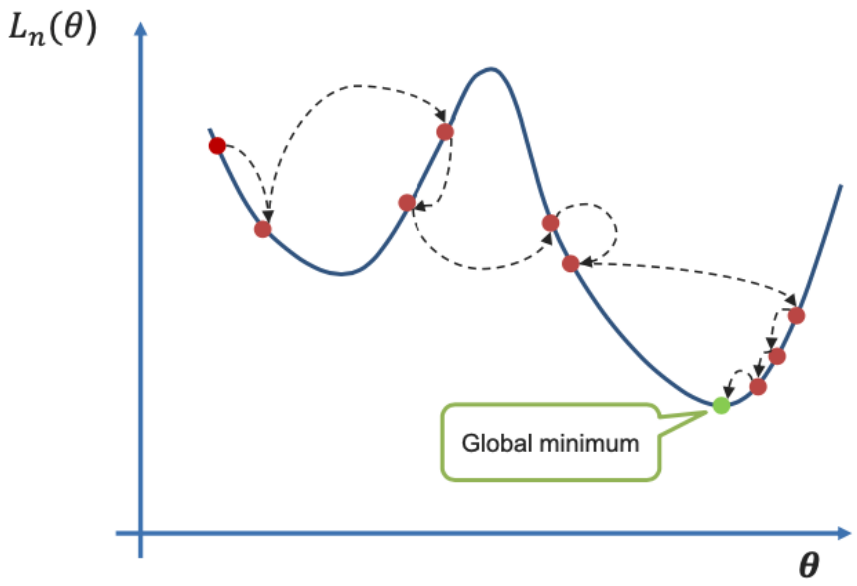
Stochastic Gradient Descent (SGD) - I

- Stochastic gradient descent (SGD) aims at overcoming batch gradient descent drawbacks
- Instead of updating parameters using the whole training set, SGD updates parameters using a single randomly picked example (hence its stochastic feature)
- Pros:
  - The algorithm is extremely fast as very little data is used at every step
  - It is therefore possible to train models on very large datasets
  - Since examples are randomly selected, SGD avoids the risk of being stuck on a local optimum and there is a better chance to find a global minimum
- Cons:
  - Due to its random feature, the algorithm cannot settle at the minimum although it can get close to it
  - This issue can be avoided by adjusting the learning rate (learning schedule)

Stochastic Gradient Descent (SGD) - II



Standard gradient descent with local minima



SGD with local minima

Stochastic Gradient Descent (SGD) - III

- SGD algorithm:
  1. Initialize $\theta^{(0)}$ elements with random values
  2. Iterate over a certain number of epochs
     1. Randomly shuffle the dataset
     2. For all $i \in [1, n]$ and for all $j \in [1, p]$

$$\theta_j^{(k+1)} := \theta_j^{(k)} - \eta \frac{\partial L_n(X, y, \theta)}{\partial \theta_j} \left(x_i, y_i, \theta_j^{(k)} \right)$$

- The term epoch refers to one cycle through the full training set, i.e. after one epoch all instances in the set have been used to update parameters
- The number of epochs is another hyperparameter of the model

Mini-batch: principle

- A limitation of SGD is its relative slowness. This is due to the fact that parameters are updated for every example
- Mini-batch gradient descent overcomes this issue by computing the gradients using small random sets of examples called mini-batches
- Mini-batch is faster than SGD because it allows to take advantage of optimized matrix operations, all the more so as computations are performed using GPUs
- Another advantage of mini-batch is that the way the algorithm progresses in parameter space is less erratic, which allows to get closer to a minimum than SGD
- However, the algorithm is more prone to get stuck to a local minimum than SGD

Mini-batch: algorithm

- The algorithm is almost the same as with SGD, the exception being that gradient computations are performed on a set of examples instead of a single one
- Mini-batch algorithm:
  1. Initialize $\theta^{(0)}$ elements with random values
  2. Iterate over a certain number of epochs:
     - Randomly shuffle the dataset
     - For all $\mathcal{B} \in \mathcal{S}$ and for all $j \in [1, p]$

$$\theta_j^{(k+1)} := \theta_j^{(k)} - \eta \frac{\partial L_n(X, y, \theta)}{\partial \theta_j} \left(x_{\mathcal{B}}, y_{\mathcal{B}i}, \theta_j^{(k)} \right)$$

- The size of a mini-batch also is a hyperparameter of the model

Backward Propagation

The Problem: Computing Gradients

For logistic regression, we want to minimize:

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\sigma(w^T x_i + b)) + (1 - y_i) \log(1 - \sigma(w^T x_i + b)) \right]$$

For gradient descent, we need:

$$\frac{\partial J}{\partial w_j} \quad \text{and} \quad \frac{\partial J}{\partial b}$$

Two approaches:

- **Manual**: compute derivatives by hand (tedious, error-prone)
- **Automatic**: let the computer do it (JAX!)

Forward Pass: Computing the Loss

$$x_i, y_i$$

$$\downarrow$$

$$\boxed{z = w^T x_i + b}$$

$$\downarrow \text{forward}$$

$$\boxed{\hat{y}_i = \sigma(z)}$$

$$\downarrow$$

$$\boxed{\ell_i = -y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)}$$

$$\downarrow$$

$$J = \frac{1}{n} \sum_{i=1}^{n} \ell_i$$

Forward pass: compute $J(w, b)$ step by step

Backward Pass: Computing the Gradient

$$\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b}$$

$$\boxed{\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} \cdot \sigma'(z)}$$

$$\boxed{\frac{\partial J}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}}$$

backward

$$\boxed{\frac{\partial J}{\partial \ell_i} = \frac{1}{n}}$$

$$J = \frac{1}{n} \sum_{i=1}^{n} \ell_i$$

Backward pass: go backward applying the **chain rule** $\frac{\partial J}{\partial w_j} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial w_j}$.

The Chain Rule

If we have a composite function $J = f(g(h(w)))$, then:

$$\frac{dJ}{dw} = \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dw}$$

Example for our logistic regression:

$$\begin{aligned}
w &\to z = w^T x + b \\
z &\to \hat{y} = \sigma(z) \\
\hat{y} &\to \ell = -y \log(\hat{y}) - (1-y)\log(1-\hat{y}) \\
\ell &\to J = \frac{1}{n} \sum \ell_i
\end{aligned}$$

Therefore:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \ell} \cdot \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Manual Backward Propagation vs JAX

Manual approach (tedious):

$$\frac{\partial J}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(\sigma(w^T x_i + b) - y_i) \cdot x_{ij}$$

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\sigma(w^T x_i + b) - y_i)$$

With JAX (automatic):

```
from jax import grad
def cost(params, X, y):
    w, b = params
    z = jnp.dot(X, w) + b
    probs = sigmoid(z)
    return -jnp.mean(y*jnp.log(probs) +
                     (1-y)*jnp.log(1-probs))
# Automatic gradient!
loss_grad = grad(cost)
```

How JAX Computes the Gradient

JAX traces the computation graph during the forward pass:

1. **Forward**: JAX executes `cost(params, X, y)` and records all operations
2. **Graph**: JAX builds a dependency graph
3. **Backward**: JAX traverses the graph backward, applying the chain rule automatically
4. **Result**: $\nabla J(w, b)$ computed exactly!

Advantages:

- No calculation errors
- Works for any differentiable function
- Very efficient (JIT compilation)

Backward Propagation in JAXopt LBFGS

In our JAXopt code, the gradient is computed automatically:

1. **We only define the cost function:**

```
def cost_jaxopt(params, X, y, C):
    return log_loss + l2_reg
```

2. **LBFGS automatically calls JAX for the gradient:**
   - At each iteration, LBFGS needs ∇J
   - JAX performs the backward pass automatically
   - LBFGS uses this gradient for the Hessian approximation

Result: We only write the cost function, JAX handles the rest!

Comparison: What We Need to Code

Scipy (manual)

- Code `cost()`
- Code `cost_grad()`
  $\rightarrow$ derivatives by hand!
- Pass both to `minimize()`

Code: ~50 lines

JAXopt (automatic)

- Code `cost()`
- JAX computes gradient automatically
- Pass only `cost` to LBFGS

Code: ~20 lines

JAX = less code, fewer errors, more flexibility!

Summary

Backward propagation:

- Technique to compute gradients efficiently
- Traverses the computation graph backward applying the chain rule
- Essential for optimization (gradient descent, LBFGS, etc.)

JAX:

- **Automatic differentiation**: computes gradients for you
- Works on any Python/NumPy function
- Used by JAXopt for LBFGS

In our logistic regression:

- We only write the cost function
- JAX performs backward pass automatically
- LBFGS optimizes using these gradients

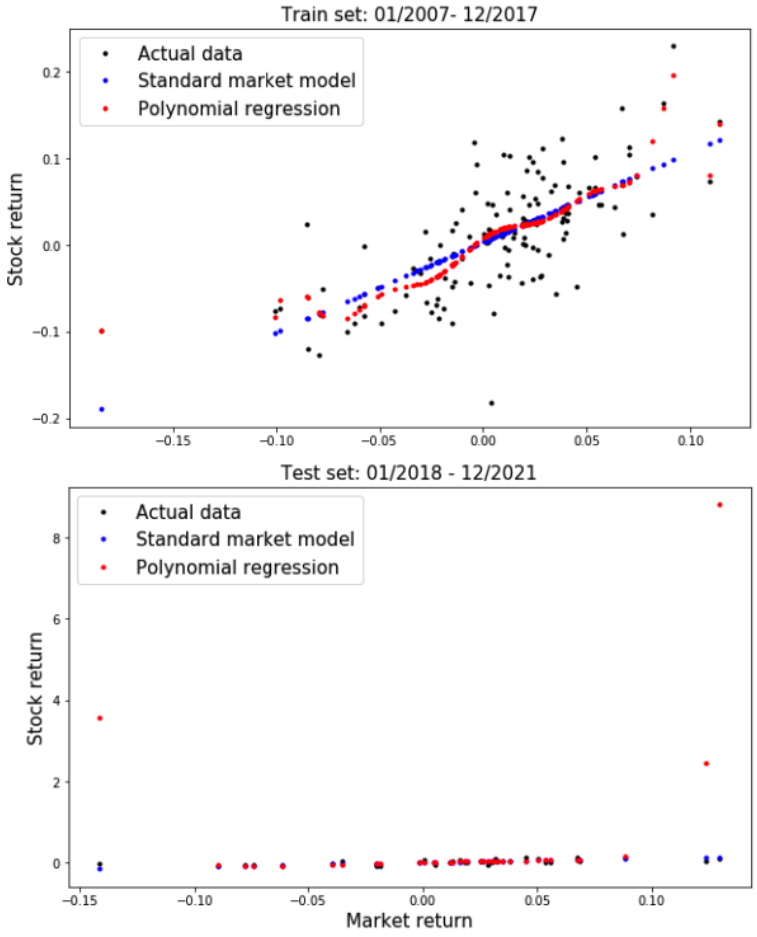
Generalization, overfitting and regularization

Generalization

- Minimizing the training loss on a given dataset implies that the algorithm will (normally) achieve minimal empirical risk
- No guarantee however that the model will make correct predictions on new (unseen) data, especially if the cost function is not well suited to the problem
- Generalization refers to the ability of a model to make accurate predictions on a new dataset, i.e. on a dataset that has not been used to train the model

Overfitting - I

- Overfitting refers to the situation where a model makes accurate predictions on the train dataset but generalizes poorly
- An overfitting example is shown in the right-hand side graphs
  - The objective is to build a model that predicts the monthly return $R_{i,t}$ of stock from the contemporaneous return $R_{m,t}$ of the market (Sharpe, 1963)
  - Black dots report the actual $(R_{m,t}, R_{i,t})$ pairs. Blue dots report the predicted $R_{i,t}$ from the standard market model $R_{i,t} = \alpha + \beta R_{m,t}$, while red dots report predicted $R_{i,t}$ from model $R_{i,t} = \alpha + \sum_{j=1}^{20} \beta_j (Rm, t)^j$



Overfitting - II

- Overfitting can be the outcome of a poor choice regarding the cost function that is used to train the model
- Overfitting may result also from issues with training data:
  - Measurement error: lack of precision in the measurement of the features
  - Label error (teacher's noise): the humans/algorithms in charge of labelling the data report wrong labels
  - Important features that explain the labels are not accounted for
  - Outliers
- In general, overfitting arises when overly complex models are used. These models excessively focus on fitting the data, including the noise they contain
  $\Rightarrow$ fitting is different from learning
- Opposite to that, too simple a model can result in underfitting

Excess risk

- To get a better understanding of the risk of a model, we can compare its risk with $\mathcal{R}^*$, where $\mathcal{R}^*$ denotes the **risk of the best possible model**. This allows computing the excess risk of the model
- Excess risk can be decomposed as follows:

$$\mathcal{R}(h^*) - \mathcal{R}^* = \left[\mathcal{R}(h^*) - \min_{h \in \mathcal{H}} \mathcal{R}(h)\right] + \left[\min_{h \in \mathcal{H}} \mathcal{R}(h) - \mathcal{R}^*\right] \qquad (2)$$

- The first bracket of the right-hand side in (2) quantifies the distance between model h^* and the optimal model in $\mathcal{H}$. It corresponds to the estimation error
- The second bracket of the right-hand side in (2) captures the quality of the optimal model in $\mathcal{H}$, i.e. the quality of the choice that has been made in the hypothesis space. It corresponds to the approximation error

Approximation error vs. estimation error

In practice, it is not possible to compute (2). Yet this representation highlights the trade-off one faces when building a model:

- Choosing a larger set of hypotheses should result in lower approximation error since it increases the likelihood to select a model that is closer to the ground truth
- However, the set being larger, the optimal solution is harder to identify

Example:

- When modeling the relationship between $R_{i,t}$ and $R_{m,t}$, we first focused on a linear model. Getting the optimal h is straightforward as OLS give directly the solution. The estimation error is thus small but comes at the cost of a larger approximation error
- We then extended the hypothesis space by considering a polynomial h. Determining the optimal degree of h, i.e. finding $\underset{h \in \mathcal{H}}{min} \mathcal{R}(h)$ where $\mathcal{H}$ is now the space of all polynomial functions, is more difficult. The approximation error is likely to be smaller than with a linear h but the estimation error increases

Model risk

- Let assume there exists a function $\phi : \mathcal{X} \mapsto \mathcal{Y}$ such that $Y = \phi(X) + \epsilon$, where η is a noise term whose variance is σ^2
- The best possible model is ϕ, where ϕ represents the true relationship between X and Y. Therefore $\mathcal{R}^* = \mathcal{R}(\phi) = \sigma^2$
- Let's compute now the risk of model h^* using quadratic cost:

$$\begin{aligned}\mathcal{R}(h^*) &= \mathbb{E}\left[(Y - h^*(X))^2\right] = \mathbb{E}\left[(\phi(X) + \epsilon - h^*(X))^2\right] \\ &= \mathbb{E}\left[(\phi(X) - h^*(X))^2\right] + \mathbb{E}\left[\epsilon^2\right] + 2\mathbb{E}\left[(\phi(X) - h^*(X))\epsilon\right] \\ &= \mathbb{E}\left[(\phi(X) - h^*(X))^2\right] + \mathcal{R}(\phi)\end{aligned}$$

- In the above:
  - Excess risk $\mathcal{R}(h^*) - \mathcal{R}(\phi)$ is equal to $\mathbb{E}\left[(\phi(X) - h^*(X))^2\right]$. It corresponds to the reductible error, i.e. the error that can be reduced by selecting the appropriate h
  - $\mathcal{R}(\phi)$ is known as the irreductible error and is due to noise

Bias-variance tradeoff - I

- ϕ is the true function that links Y to $X \Rightarrow$ whatever $\mathcal{S}$ is used, ϕ will always be the same
- On the contrary, h^* will change depending on the $\mathcal{S}$ set that is used to train the model
- Let denote $h^*(x)$ the prediction we get for a given x using dataset $\mathcal{S}$, and let denote $\mathcal{E}\left[h^*(x)\right]$ the average prediction we would get if the model were trained on a very large number of datasets
- As the true function does not depend on $\mathcal{S}$, $\mathbb{E}\left[\phi(x)\right] = \phi(x)$

Bias-variance tradeoff - II

- Now, let's decompose the reductible error $\mathbb{E}\left[(\phi(X) - h^*(X))^2\right]$
- $\mathbb{E}\left[(\phi(X) - h^*(X))^2\right] = \mathbb{E}\left[(\phi(X) - h^*(X) + \mathbb{E}(h^*(X)) - \mathbb{E}(h^*(X)))^2\right]$
- The new expression can be rewritten as the sum of the following terms:

$$\mathbb{E}\left[(\phi(X) - \mathbb{E}(h^*(X)))^2\right] \tag{3}$$

$$+\mathbb{E}\left[(\mathbb{E}(h^*(X)) - h^*(X))^2\right] \tag{4}$$

$$+2\mathbb{E}\left[(\mathbb{E}(h^*(X)) - h^*(X))\,(\phi(X) - \mathbb{E}(h^*(X)))\right] \tag{5}$$

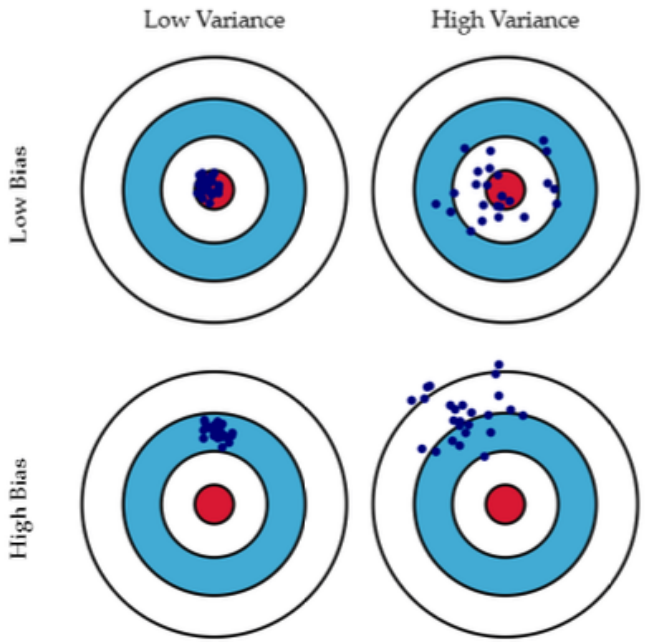
- As $\phi(x)$ and $\mathbb{E}\left[(h^*(X))\right]$ are constants, (3)$= [\phi(x) - h^*(x)]^2$
- By the same token, $\phi(x) - \mathbb{E}\left[h^*(x)\right]$ can be taken out of the expectation in (5), so that
  (5)$= 2 \times [\phi(x) - \mathbb{E}\left[h^*(x)\right]] \times \mathbb{E}\left[\mathbb{E}\left[h^*(x)\right] - h^*(x)\right]$
- As the expectation of $\mathbb{E}\left[h^*(x)\right] - h^*(x)$ is zero, (5) $= 0$

Bias-variance tradeoff - III

Therefore, the reductible error can be decomposed as follows:

$$\begin{aligned}\mathbb{E}\left[(\phi(X)-h^*(X))^2\right] &= [\phi(x)-\mathbb{E}(h^*)]^2 + \mathbb{E}\left[(\mathbb{E}(h^*(x))-h^*(x))^2\right] \\ &= \mathrm{Biais}^2\left[h^*(x)\right] + \mathrm{Var}\left[h^*(x)\right]\end{aligned}$$

- The bias of a model quantifies the extent to which the predicted labels are different from the true labels. Bias captures the inherent error of the model (even with infinite training data) because the model is "biased" towards a particular kind of solution
- The variance of a model captures how much a model changes if it is trained on a different training set. A high variance model is over-specialized to a training dataset



Source: http://scott.fortmann-roe.com/docs/BiasVariance.html

Regularization - I

- The simpler a model, the lower it is prone to overfit the data. Regularization is a technique aimed at reducing the complexity of a model
- A regularized model includes additional terms that penalize complex models
- The most common regularization techniques are Ridge and Lasso (Least Absolute Shrinkage and Selection Operator)
  - The Ridge penalty or L2 penalty aims at keeping the model weights (or parameters) as small as possible
  - The Lasso penalty or L1 penalty narrows down the number of non-zero parameters – Combining linearly the L1 and the L2 penalties yields the Elastic Net regularization
- With regularization, a simple model can be preferred over a complex one even if the former displays larger errors/bias on the training set
- Put differently, regularization can be viewed as a solution to the bias/variance tradeoff

Regularization - II

- Training loss function for L2 or Ridge regularization:

$$L_n(X, y, \theta)_{L_2} = L_n(X, y, \theta) + \lambda \sum_{j=1}^{p} \theta_j^2$$

- Training loss function for L1 or Lasso regularization:

$$L_n(X, y, \theta)_{L_1} = L_n(X, y, \theta) + \lambda \sum_{j=1}^{p} |\theta_j|$$

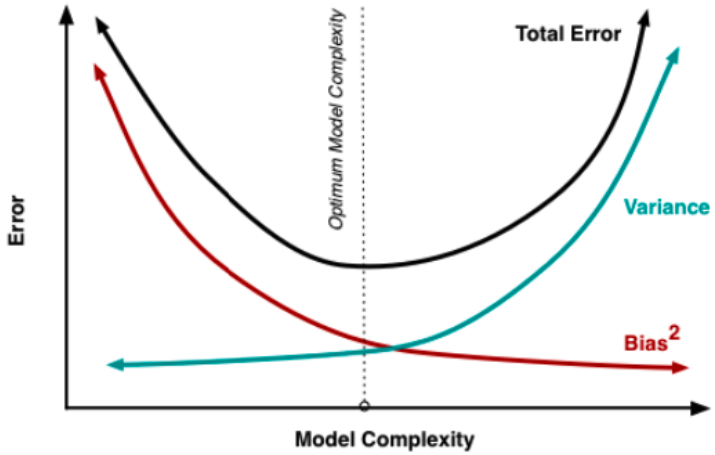
- Training loss function for Elasticnet regularization :

$$L_n(X, y, \theta)_{EN} = L_n(X, y, \theta) + \alpha \sum_{j=1}^{p} \left[(1-\lambda)\theta_j^2 + \lambda |\theta_j|\right]$$

- Both α and λ are model hyperparameters. If λ is set to 0 (1), Elastic Net reduces to the L2 (L1) penalty

Detecting high bias and high variance

- High variance:
  - Symptoms:
    - The error of the model on training set $\mathcal{S}$ is much lower than on validation set
    - The error on the training set is lower than the desired threshold error ξ while the error on the validation set is above ξ
  - Remedies:
    - Add more training data points / simplify the model/use Bagging (more on this in next slides)
- High bias:
  - Symptoms:
    - Training error is larger than ξ
  - Remedies:
    - Use a more complex model / add features / use Boosting (more on this in next slides)



Source: http://scott.fortmann-roe.com/docs/BiasVariance.html

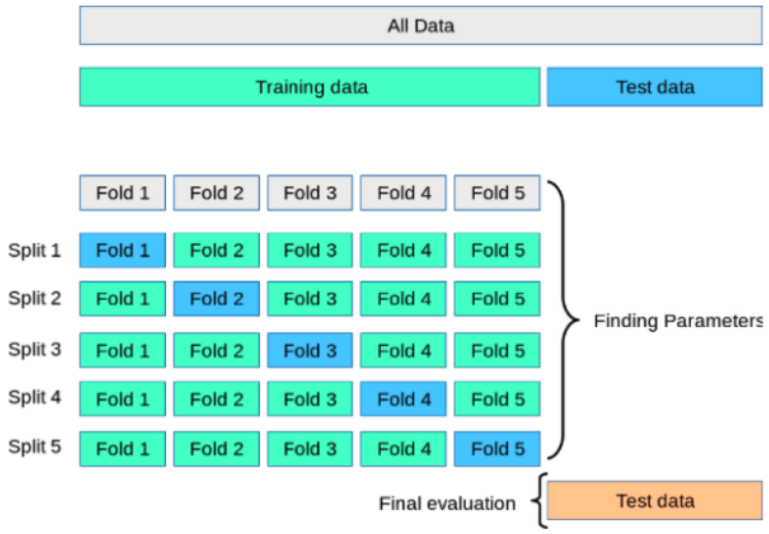
Model selection

Framework

- Suppose we plan to explore M candidate algorithms
- The first step is to train the M algorithms on the train set to come up with first pass models $h_1^*, h_2^*, \ldots, h_M^*$
- The second step is to fine-tune the models using cross-validation or bootstrap
- The third step is to evaluate the models on the validation set in order to select the final model based on performance metrics (see next slides)
- The last step is to run the final model on the test set in order to gauge its ability to generalize

Cross-validation - I

- In k-fold cross-validation (CV), the training set is randomly split into k smaller sets
- Then the following procedure is applied for each of the k folds:
  1. The algorithm is trained using $k-1$ of the folds as training data
  2. The resulting model is validated on the remaining part of the data, i.e. the remaining k^{th} fold is used as a pseudo test set to compute performance scores (see next slides)



Source: https://scikit-learn.org/stable/modules/cross_validation.html

- For classification problems, stratified CV is applied to ensure that the proportion of a given class in a fold is the same as in the whole training set

Cross-validation - II

- CV is used to analyze how a model generalizes to unseen data
  - Comparing the average error of the model on the validation folds with the error on the training folds allows to spot variations in the error and potential overfitting
  - One possibility is to compute the variance of the errors across the folds to get an idea of the variance of the model
- CV is used also to fine-tune a model
  - A model involves several hyperparameters
  - Analyzing how a model performs on validation folds depending on its hyperparameters allows it to adjust their values and tweak the model
  - The first technique to determine optimal hyperparameters is grid search: the modeller defines the list of the H_i values to be tested for each hyperparameter i, and the model is trained with each of them. Computational cost can be high as the number of tested models is equal to $\prod_{i=1}^{I} H_i$
  - The second technique is random search: a range of values is defined for each hyperparameter and random values are selected within these ranges

Bootstrap

- Bootstrap means random sampling with replacement
- Bootstrap is used frequently to derive the sampling distribution of a statistic
- In ML, bootstrap is mostly used in the context of ensemble learning to aggregate the predictions of several models or algorithms
- Bootstrap also can be an altervative to cross-validation
- In that case, bootstrap is used to build random training sets where each training set is built from as many random draws (with replacement) as the number of examples n in training set $\mathcal{S}$
- It can be shown that on average, for a large n, 36.8% of the observations will not be drawn. This implies that, on average, for each bootstrapped training set, 63.2% of the examples can be used for training and 36.8% of them can be used as a validation set

Performance metrics

Loss vs. performance

- A model always achieves the lowest training loss conditional on its underlying hypothesis and the optimization algorithm that is used
- However, achieving the best model according to the training loss criterion does not necessarily imply that the outputs of the model are in line with the expectations of its user
- Performance metrics are needed to determine which model should be chosen
- Example: shoplifting detection (Géron, 2019)
  - Suppose a classification model has been trained to detect shoplifters on surveillance images
  - Which model would you prefer?
    - A model that identifies 100% of the shoplifters but such that 50% of the alleged shoplifters haven't stolen items
    - A model that identifies 50% of the shoplifters and is 100% correct when it identifies a shoplifter

Performance metrics for classification

Confusion matrix

- The confusion matrix is a table that reports the number of true positives, true negatives, false positives and false negatives
- For binary classification, the structure of the confusion matrix is as follows (Scikit-learn display):

True label			
	0	TN	FP
	1	FN	TP
		0	1
		Predicted label	

- For multiclass problems the number of rows and columns of the matrix are equal to the number of classes

Performance metrics - I

Based on confusion matrix, it is possible to compute various scores that help understand how a classifier behaves

- Accuracy: fraction of correctly classified examples

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- Precision: fraction of correct predictions when a classifier assigns an example to class 1

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- Recall (True Positive Rate – TPR or sensitivity): percentage of class 1 examples the classifier is able to identify

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Performance metrics - II

Based on confusion matrix, it is possible to compute various scores that help understand how a classifier behaves

- F1 score: harmonic mean of precision and recall

$$\mathrm{F1} = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}}$$

- Specificity: fraction of class 0 examples identified among all class 0 examples

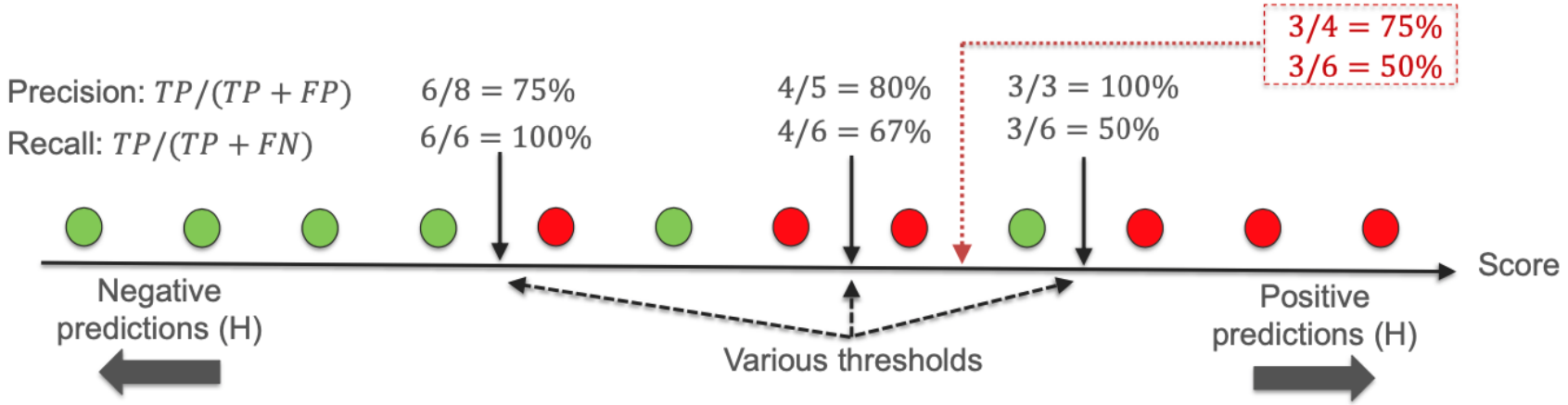
$$\text{specificity} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$$

- False Positive Rate (FPR = 1 specificity): fraction of class 0 examples that are incorrectly classified as class 1 examples

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}$$

Precision/Recall tradeoff

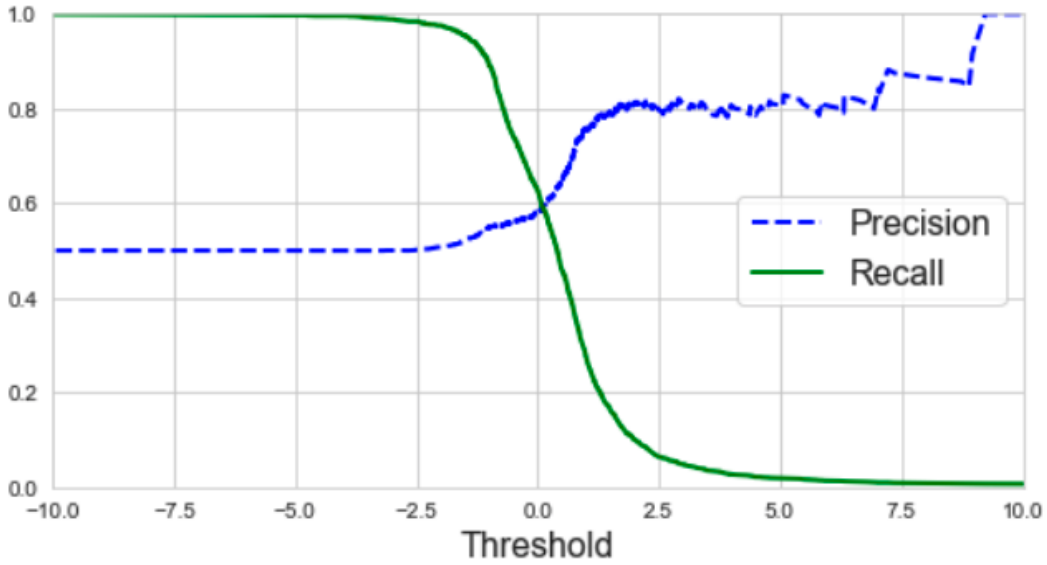
- A binary classifier assigns an example to one of the two classes based on a decision function
  - For each instance the classifier computes a decision score $\hat{s}_i = x_i\hat{\theta}$
  - Denoting T a threshold value, if $\hat{s}_i \geq T(\hat{s}_i < T)$, example i is classified as being from class 1 (0). By default, the threshold value is set to 0 (50% threshold)
  - Varying the threshold value impacts the way the instances are classified, and therefore precision and recall scores



Source: A. Géron, "Hands-on Machine Learning with Scikit Learn, Keras & TensorFlow", 2nd ed., O'REILLY (2019)

Precision vs recall: example

- It is possible to get the precision/recall tradeoff of a classifier by computing the value of the two metrics for different threshold levels



- NOTE: recall always decreases with threshold value whereas the evolution of precision is non-monotonic (see the previous slide for an illustration of why precision can decrease when threshold increases)

ROC curve and AUC - I

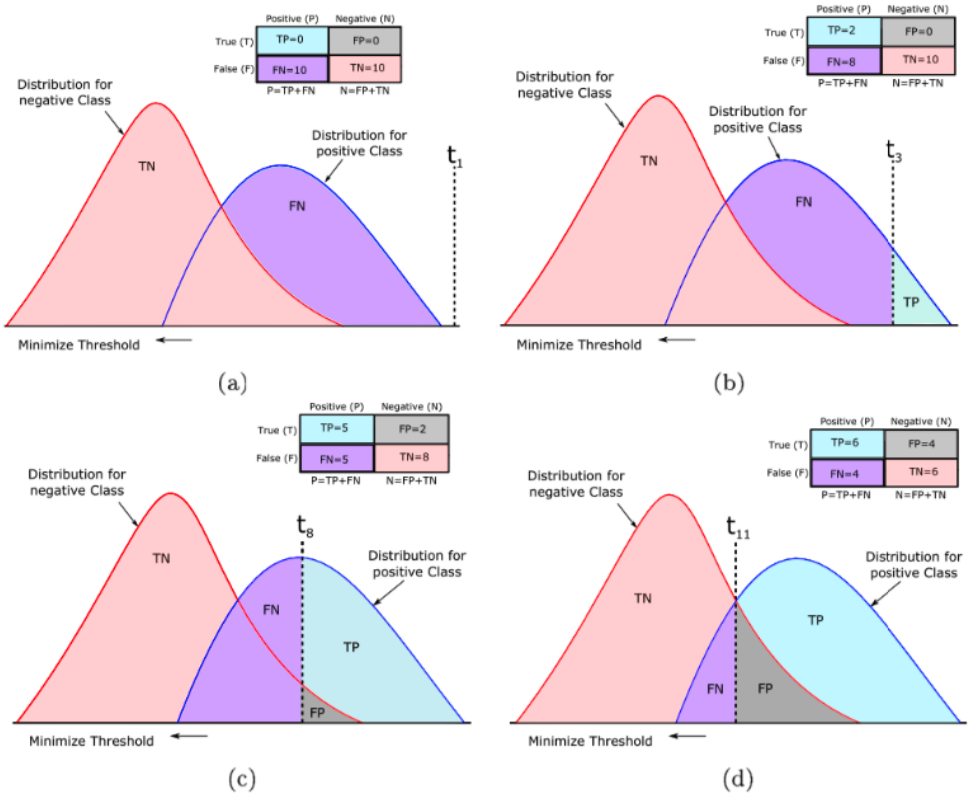
- The Receiver Operating Characteristic (ROC) curve is a plot that displays the diagnostic ability of a binary classifier when the threshold is varied
- It is created by plotting the true positive rate (TPR) or Recall against the false positive rate (FPR), i.e. 1 – specificity, at various threshold settings
- The ROC curve also can be thought of as a plot of the power (probability of detecting a true positive) against the Type I error (wrongly assigning a class 0 example to class 1)
- Very often the Area Under the ROC Curve or AUC is used to compare models and to select the best one (the one with the highest AUC)

ROC curve and AUC - II

TP = 0	FP=0
FN = 10	TN=10

$TPR = 0$

$FPR = 0$



TP = 2	FP=0
FN = 8	TN=10

$TPR = \frac{2}{2+8} = 0.2$

$FPR = 0$

TP = 5	FP=2
FN = 5	TN=8

$TPR = \frac{5}{5+5} = 0.5$

$FPR = \frac{2}{2+8} = 0.2$

TP = 6	FP=4
FN = 4	TN=6

$TPR = \frac{6}{6+4} = 0.6$

$FPR = \frac{4}{4+6} = 0.4$

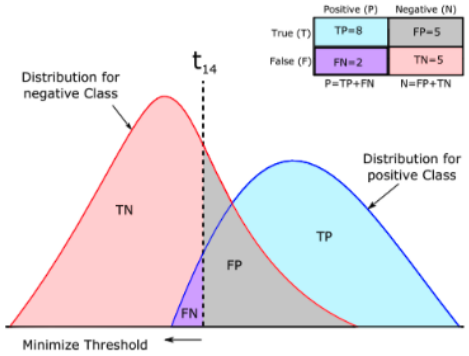
Source: A. Tharwat, "Classification assessment methods", Applied Computing and Informatics (2020)

ROC curve and AUC - III

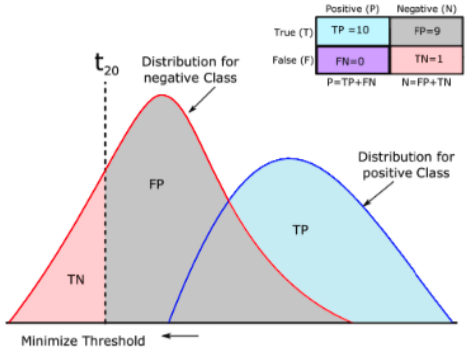
TP = 8	FP=5
FN = 2	TN=5

$$TPR = \frac{8}{8+2} = 0.8$$

$$FPR = \frac{5}{5+5} = 0.5$$



(e)

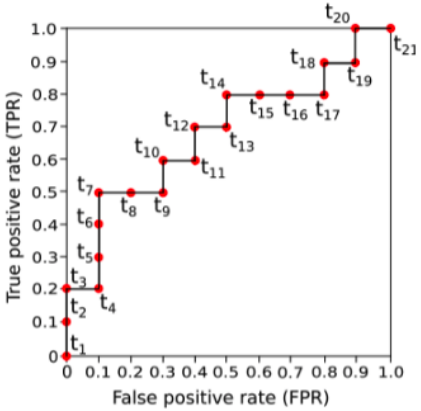


(f)

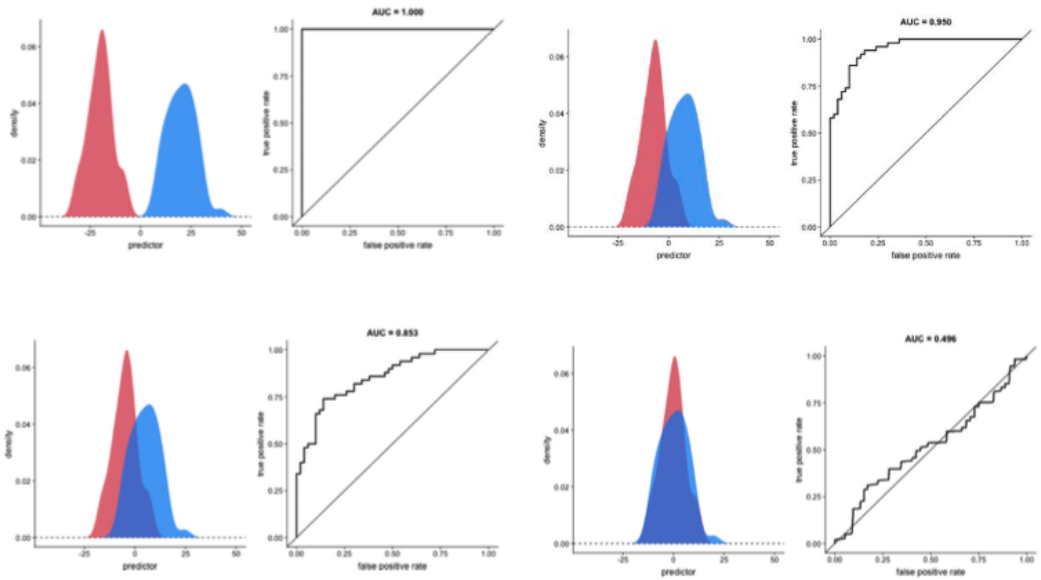
TP = 10	FP=9
FN = 0	TN=1

$$TPR = \frac{10}{10+0} = 1$$

$$FPR = \frac{9}{9+1} = 0.9$$



ROC curve and AUC - IV



Source: https://paulvanderlaken.com/2019/08/16/roc-auc-precision-and-recall-visually-explained/

- Formally, the AUC is equal to the probability that the decision function f assigns a higher value to a randomly drawn positive example x^+ than to a randomly drawn negative example x^-, i.e., $\mathrm{AUC}(f) = \Pr(f(x^+) > f(x^-))$
- In a nutshell, the ROC AUC indicates how well the classifier separates the probabilities of the positive classes from those of the negative classes

Performance metrics for regression

MSE and RMSE

- For regression, the main performance metrics are MSE and RMSE (Root Mean Squared Error)

$$\mathrm{MSE} = \frac{1}{n}\left[y - h_\theta^*(X)\right]^T \left[y - h_\theta^*(X)\right]$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\left[y - h_\theta^*(X)\right]^T \left[y - h_\theta^*(X)\right]}$$

- The advantage of RMSE over MSE is that RMSE allows to express the error in the same units as those of the labels

R^2

- The R^2 corresponds to the proportion of the variance of the labels that is explained by the model. Its expression is:

$$\frac{\left[y - h_\theta^*(X)\right]^T \left[y - h_\theta^*(X)\right]}{\left[y - \frac{1}{n} y^T \mathbb{I}\right]^T \left[y - \frac{1}{n} y^T \mathbb{I}\right]}$$

  where $\mathbb{I}$ denotes the column vector of 1s.
- By construction, $\mathrm{R}^2 \in [0, 1]$. The higher the R^2, the higher the goodness of fit of the model
- With simple regression (1 feature), R^2 is the square of the correlation coefficient between x and y

Next steps

Validation and test

- Once all candidate models have been evaluated on the validation set using the appropriate performance metrics, the next step is to choose the optimal model
- Finally, the selected model is tested on the test set. Fingers crossed...
- Keep in mind:
  - In general the performance on the train set will be below that on the train and validation sets
  - **Golden rule:** you must never tweak the final model after seeing its behaviour on the test set

ML and Ethics - Different perspectives

1. **Privacy**: Data are owned by users ; they might be used collectively (estimation, inference) but cannot be traced back to specific user
2. **Fairness**: Users are different but should be treated equally; some features are sensible and should not impact the outcome of an algorithm
3. **Security**: Users are willing to send and share data; yet outsiders should not be able to decypher them Related to encryption (homomorphic ?)

ML and Ethics - Typical techniques

- **Privacy**: data of a specific user can-not be "recovered"
  - Replace user data by uniform noise with small proba
  - Add (Laplace) noise to the output.
- **Fairness:** sensitive data of a user should not influence algo
  - Many other definitions, all incompatible with each other
  - Separation achieved by post-processing (ROC curves)
- **Privacy** and **Fairness** not always compatible

$\Rightarrow$ Nice Dauphine podcast:
https://dauphine.psl.eu/podcasts/ex-machina

Explainable AI (XAI)

Increasing Performance through Complexity:

- Why Explainability Matters?
- How do we make people comfortable with the model?

$\Rightarrow$ Explainable AI (XAI)

Typical techniques:

- Shapley Values (SHAP)

Reproductibility

Great course on that by Data Lab of Insee:
https://ensae-reproductibilite.netlify.app/introduction/
Slides:
https://avouacr.github.io/ensae-reproductibilite-slides/

Have fun with Machine Learning!